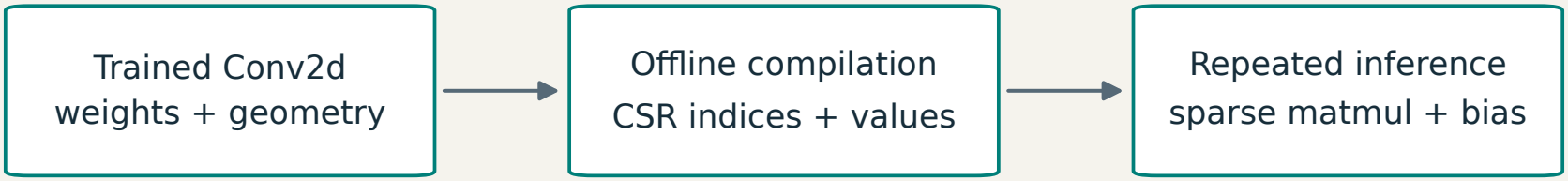
Train once. Change the execution.

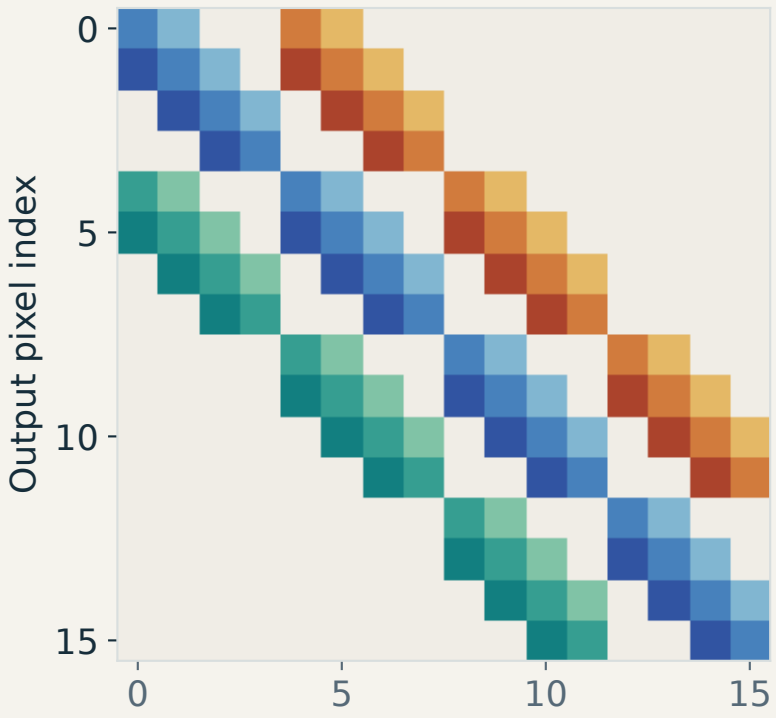
A frozen convolution is a sparse linear map at a fixed input shape.
The learned weights are reused; no retraining is required.



One 3×3 kernel



16×16 sparse operator M



Example: 4×4 input, stride 1, zero padding 1, one channel.

Same color = the same trained weight, repeated in M.

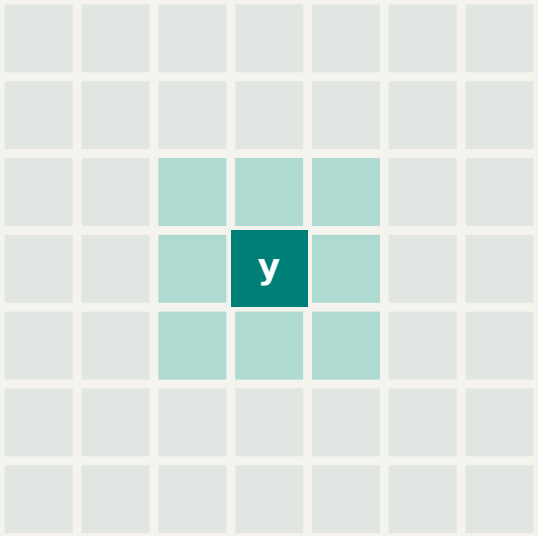
$$Y_{flat} = MX_{flat} + b$$

Each batch sample is a column of X.

Two ways to execute the stencil.

The CUDA kernels compute the same weighted neighborhoods directly.
They keep the kernel compact instead of expanding it into CSR.

DIRECT CUDA



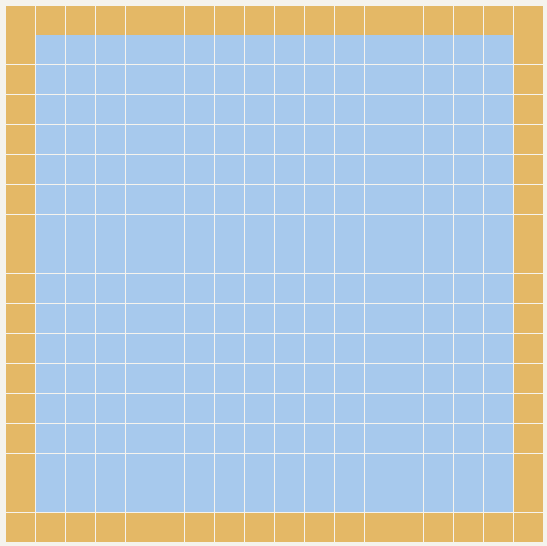
One thread → one output

Read neighbors and weights.

Accumulate channels and offsets.

Write one output value.

TILED CUDA



One block → a 16×16 tile

Load an 18×18 input patch.

Synchronize the block.

Reuse neighbors in shared memory.

Gold cells = halo for a 3×3 kernel

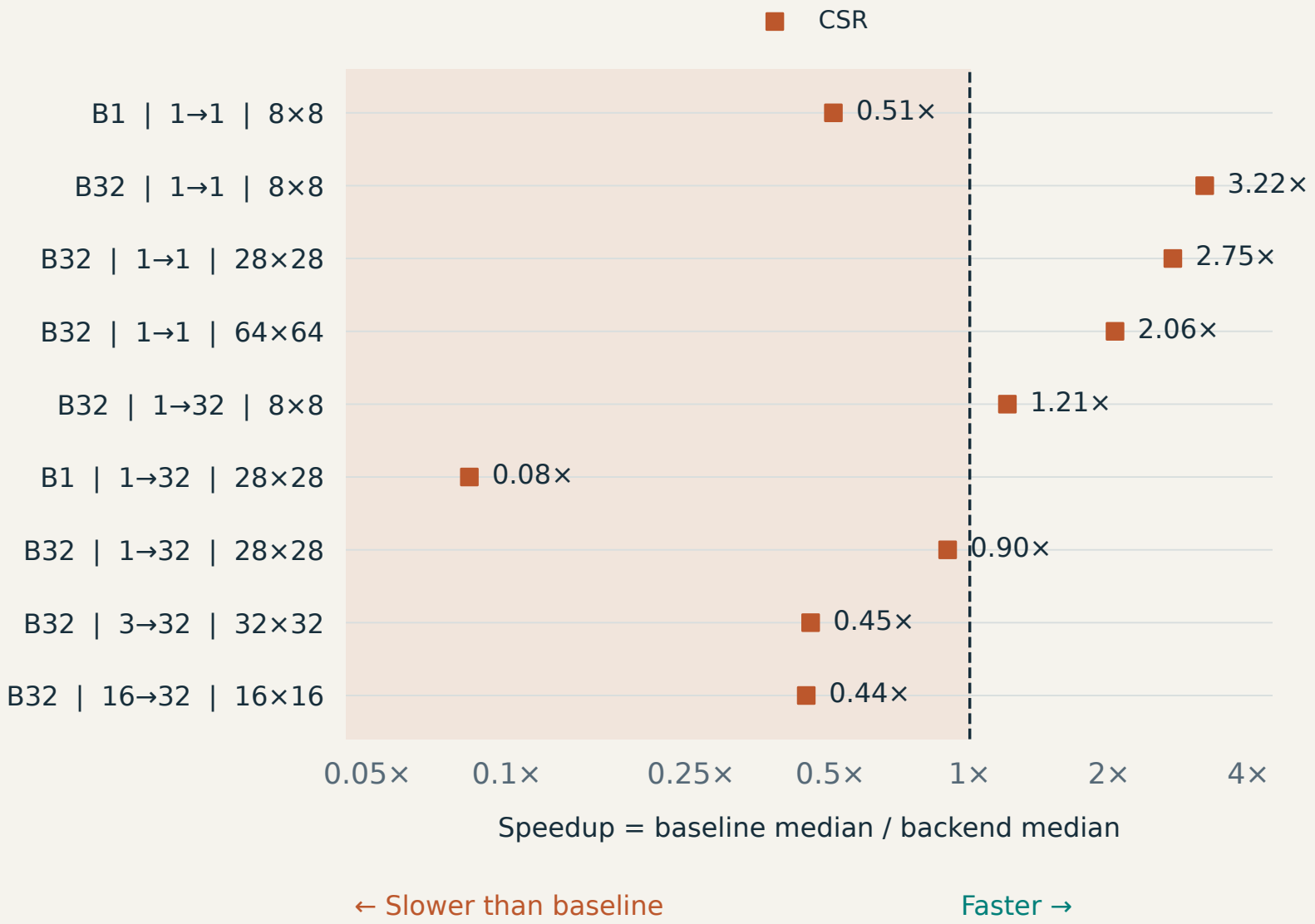
Current scope: stride 1 • dilation 1 • no groups

Tiled specialization: one input and one output channel.

Topology is implicit in the CUDA indexing; it is not a generic graph kernel.

CPU: batching changes the result.

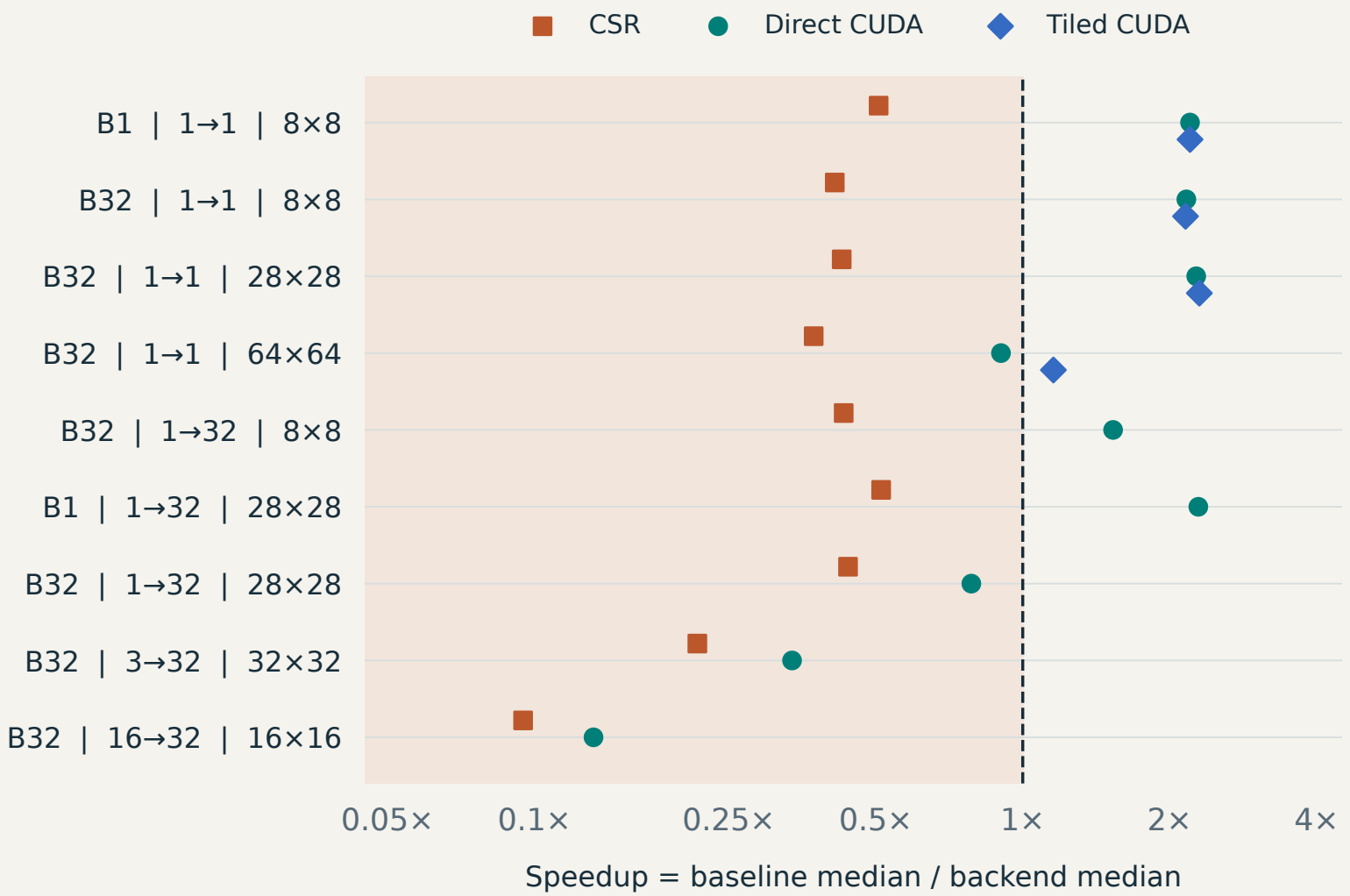
Intel i7-12700K • float32 • one CPU thread
CSR inference compared with the complete eager Conv2d call.



All 9 measured shapes shown. B = batch size; channels = input → output.
Warm-up + 9 timing groups per backend; compilation excluded.
Medians from saved benchmark JSON; ratios are not confidence intervals.

CUDA: small shapes benefit.

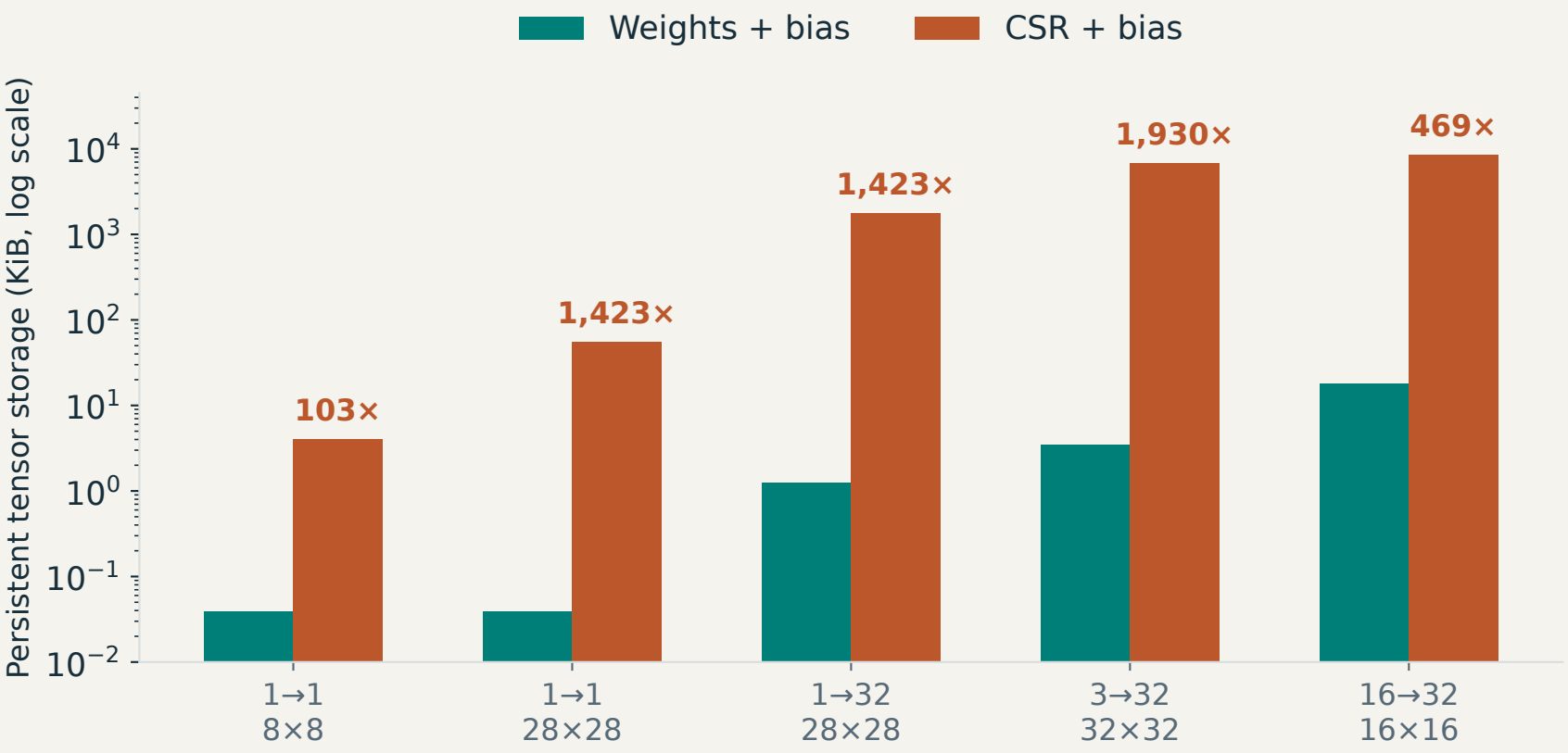
RTX 3060 • float32 • cuDNN autotuning enabled
Direct, tiled and CSR backends compared with the same Conv2d.



All 9 measured shapes shown. B = batch size; channels = input → output.
CUDA events around eager launches; host launch gaps may contribute.
Medians of 9 timing groups; hardware-specific operator timings.

The cost of explicit sparsity.

CSR stores repeated weights and indices at each valid output location.
Operator storage grows even when the trained kernel stays small.



1→32 on 28×28: 1.25 KiB → 1.74 MiB

Direct CUDA keeps the original weight tensor.

Tiled CUDA adds a temporary shared-memory patch per block.

Storage is calculated from tensor values and indices, not peak process memory.
Activations, allocator caches and backend workspaces are excluded.